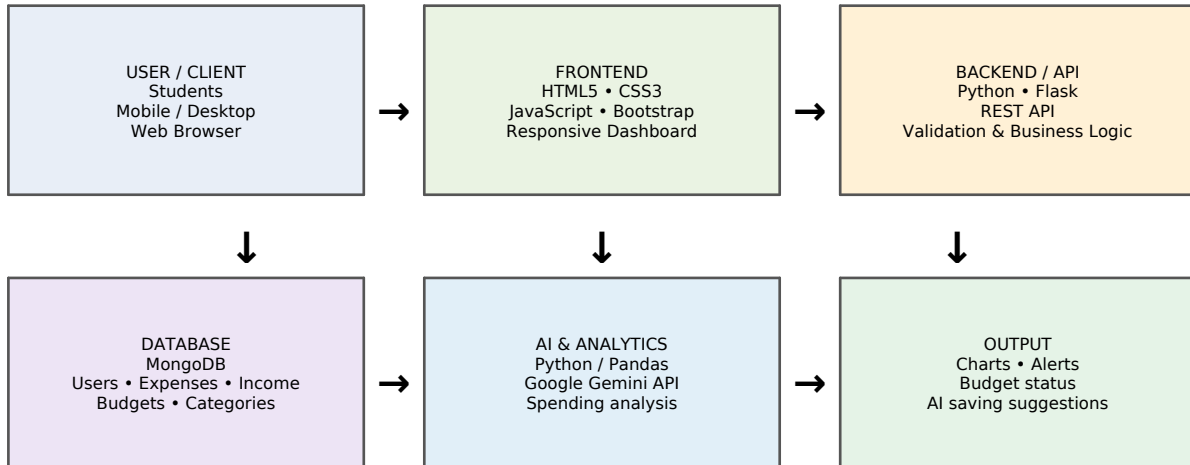


Solution Architecture

Date	29 September 2026
Team ID	SWTID-2026-2087
Project Name	POCKET SMART AI
Maximum Marks	5 Marks

POCKET SMART AI – Solution Architecture

The solution architecture shows how users, the web interface, backend services, database, AI/analytics components and the final dashboard work together as one system.



Architecture Data Flow

Step	Architecture Layer	Data / Processing Flow
1	User Request	User adds income/expense, sets a budget or requests a spending insight.
2	Frontend	The responsive web interface collects the input and sends a structured request to the backend.
3	Backend	Flask validates the request, applies business rules and communicates with the database and AI services.
4	Data Layer	MongoDB stores and retrieves user, transaction, category and budget information.
5	AI / Analytics	Python/Pandas analyses spending patterns; Gemini can generate contextual explanations and saving suggestions.
6	Presentation	The backend returns results and the dashboard displays charts, alerts, budget status and recommendations.

Technology & Component Mapping

Date	29 September 2026
Team ID	SWTID-2026-2087
Project Name	POCKET SMART AI
Maximum Marks	5 Marks

Layer	Technology	Purpose
Presentation	HTML5, CSS3, JavaScript, Bootstrap	Responsive UI, forms, dashboard and user interaction
Application	Python, Flask, REST API	Business logic, validation, authentication and API handling
Data	MongoDB	User profiles, income, expenses, budgets and categories
AI / Analytics	Python, Pandas, Google Gemini API	Spending analysis, insights and personalized saving suggestions
Visualization	Chart.js	Category breakdowns, budget progress and spending trends
Development	VS Code, Git, GitHub	Coding, version control and team collaboration
Deployment	Cloud hosting / API services	Make the application accessible and support external AI API calls

Key Architecture Principles

- Modular design: frontend, backend, database and AI services are separated for easier maintenance.
- Secure data access: authenticated users should only access their own financial records.
- API-based integration: the backend acts as the controlled bridge between the UI, database and AI service.
- Scalable foundation: additional reports, categories, alerts and AI capabilities can be added without redesigning the whole system.
- User-focused output: complex financial data is converted into simple charts, alerts and understandable recommendations.

Final Architecture Flow: User → Responsive Frontend → Flask Backend/API → MongoDB + AI/Analytics → Charts, Alerts & Personalized Insights → User Dashboard.